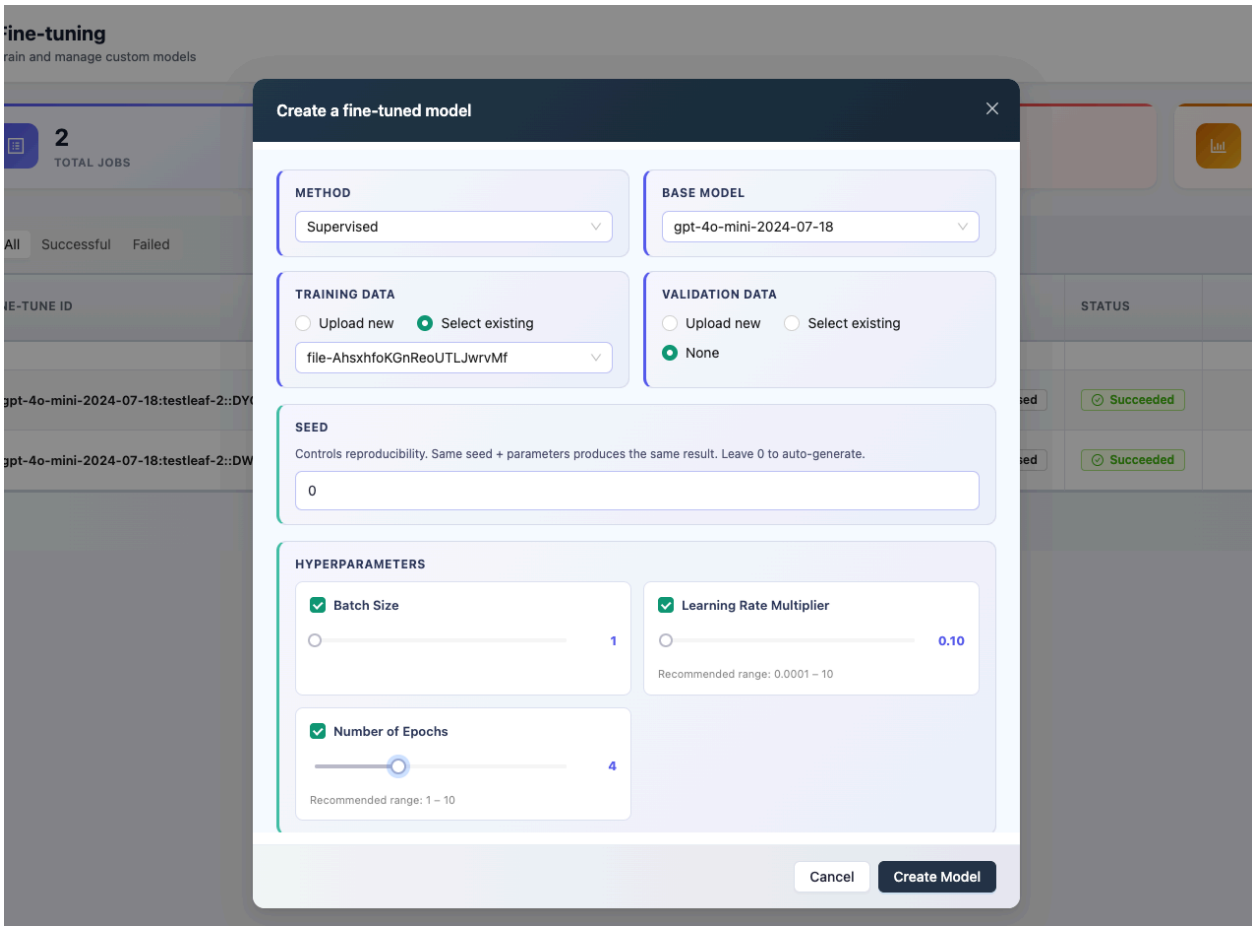
Fine Tune the model and use it in web-extension and verify the result generated based on the training.

Open https://api.testleaf.com/finetuning
Click on ‘Create’
Set the settings as below
Method: supervised fine tuning
Base model: gpt-4o-mini-2024-07-18
Training data : Upload the training_data_chat.jsonl file
HyperParameters
    Batch Size: 1
    Learning Rate Multiplier: 0.1
    Number of Epoch: 4



Click Create model.

Once Fine Tuning is done we will get a FuneTune ID

All Successful Failed

+ Create

FINE-TUNE ID	BASE MODEL	METHOD	STATUS	TRAINED TOKENS	CREATED
ft:gpt-4o-mini-2024-07-18:testleaf-2::DYQfLVqA	gpt-4o-mini-2024-07-18	supervised	Succeeded	21,316	25 Apr 2026 11:41
ft:gpt-4o-mini-2024-07-18:testleaf-2::DWD9OhA1	gpt-4o-mini-2024-07-18	supervised	Succeeded	21,316	19 Apr 2026 08:55

1–2 of 2 1 10 / page

Copy the Fine Tune ID : ft:gpt-4o-mini-2024-07-18:testleaf-2::DYQfLVqA

Open AI Extender project in VsCode

Open popup.js file and navigate to modelsByProvider method

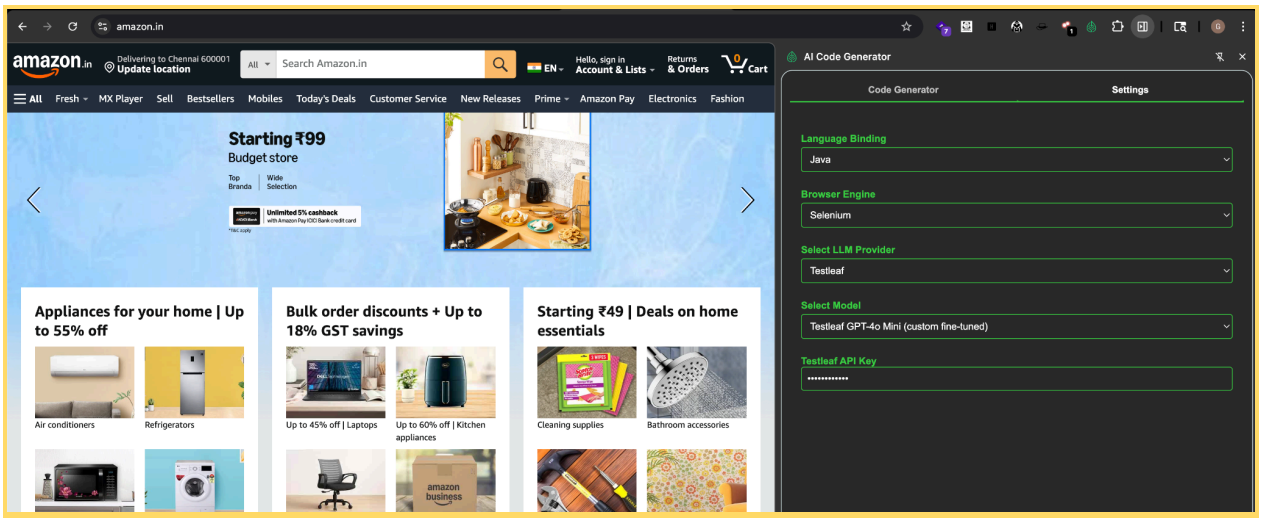
Add new component

```
testleaf: [
    {value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::DYQfLVqA', label: 'Testleaf GPT-4o Mini (custom fine-tuned)'}
  ]
```

And Save.

Refresh the extension at chrome://extensions/

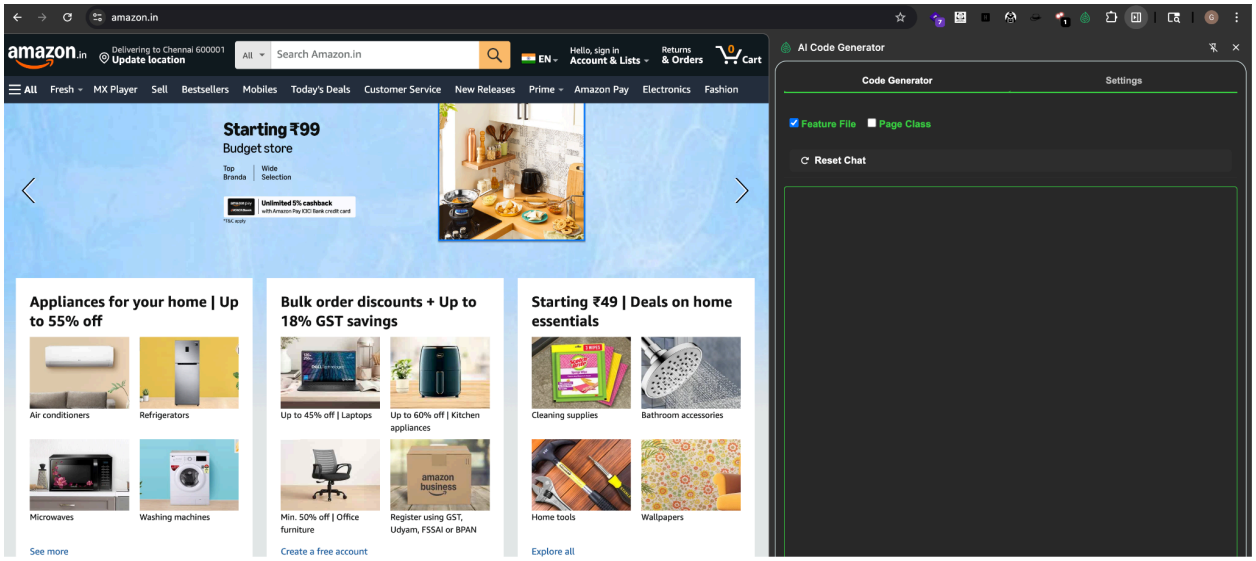
Open desired test page (amazon.in)
Click on extension icon → Settings
Select LLM Provider to Testleaf
Select Model: Testleaf GPT-4o Mini (custom fine-tuned)



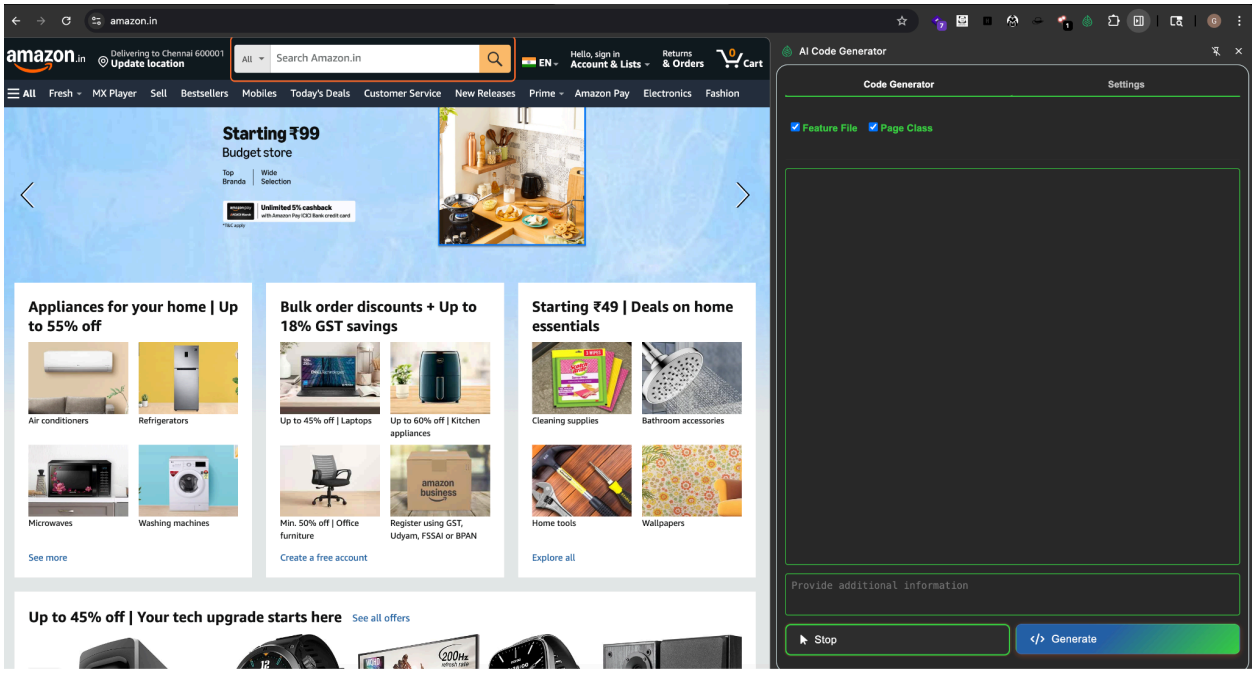
Navigate back to https://api.testleaf.com/apikey and click on create key.
Provide a name for the API key and click on Create Key

A unique key would have been generated. Copy the key and paste it in the Tesleaf API key of the browser extension.

Navigate to code generator select desired options from Feature file or Page class (Feature file and Page class)

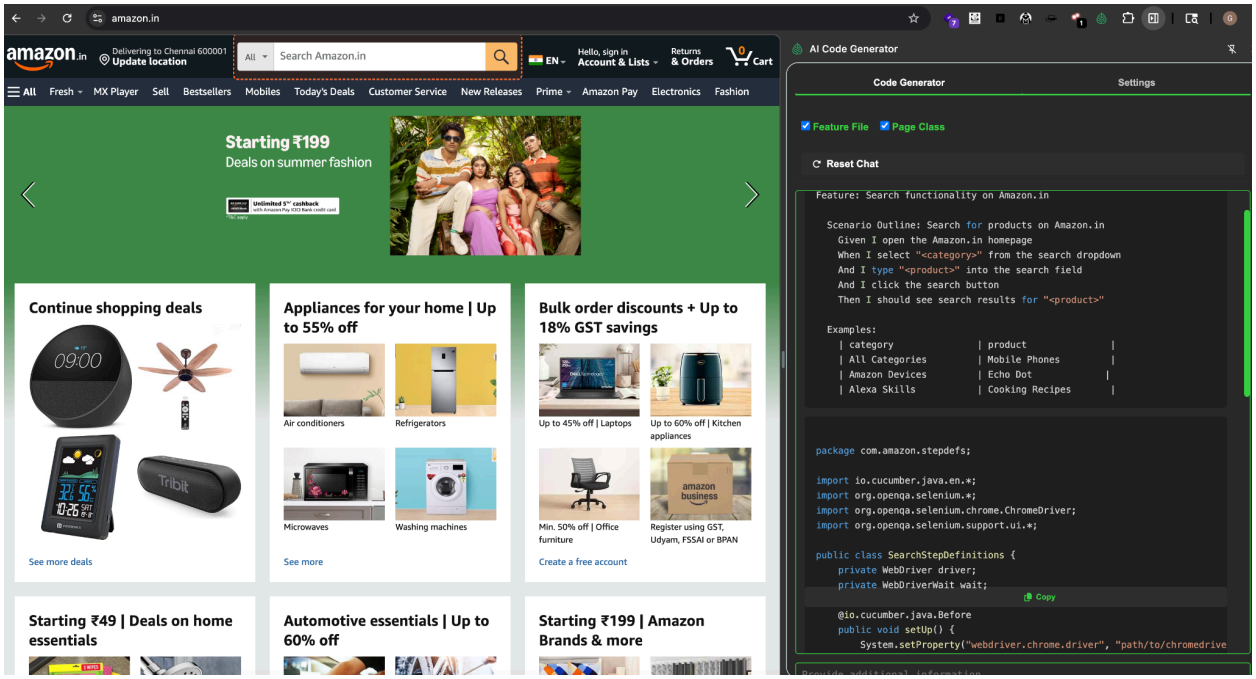


Click on Inspect and select the search bar



Click </>Generate.

Feature file and page class is generated



Code

```
Feature: Search functionality on Amazon.in

  Scenario Outline: Successful search for products
    Given I open the Amazon homepage
    When I select "<category>" from the search category dropdown
    And I type "<searchTerm>" into the search field
    And I click the search button
    Then I should see search results for "<searchTerm>"

  Examples:
    | category              | searchTerm      |
    | "All Categories"      | "laptop"        |
    | "Amazon Devices"      | "Echo Dot"      |
    | "Alexa Skills"        | "music"         |
```

Page Class

```
package com.leaftaps.stepdefs;

import io.cucumber.java.en.*;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.*;

public class SearchStepDefinitions {
    private WebDriver driver;
    private WebDriverWait wait;

    @io.cucumber.java.Before
    public void setUp() {
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver"); // Set the
path to your chromedriver
        driver = new ChromeDriver();
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        driver.manage().window().maximize();
    }

    @io.cucumber.java.After
    public void tearDown() {
        if (driver != null) driver.quit();
    }

    @Given("I open the Amazon homepage")
    public void openAmazonHomepage() {
        driver.get("https://www.amazon.in/");
    }

    @When("I select {string} from the search category dropdown")
    public void selectSearchCategory(String category) {
        WebElement dropdown =
wait.until(ExpectedConditions.elementToBeClickable(By.id("searchDropdownBox")));
        dropdown.click();
        Select select = new Select(dropdown);
        select.selectByVisibleText(category);
    }
```

```
    @When("I type {string} into the search field")
    public void enterSearchTerm(String searchTerm) {
        WebElement searchField =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("twotabsearchtextbox")))
;
        searchField.sendKeys(searchTerm);
    }

    @When("I click the search button")
    public void clickSearchButton() {
        WebElement searchButton =
wait.until(ExpectedConditions.elementToBeClickable(By.id("nav-search-submit-button"))
);
        searchButton.click();
    }

    @Then("I should see search results for {string}")
    public void verifySearchResults(String searchTerm) {
        WebElement results =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("span.a-color-
state.a-text-bold")));
        String resultsText = results.getText();
        assert resultsText.contains(searchTerm) : "Expected search results to contain: " +
searchTerm + ", but got: " + resultsText;
    }
}
```